Select a definition **MUSTLink User Manual**

MUSTLink REST API 1.6.2

[Base URL: /ng-webmust/mustlink/]

https://plato.esac.esa.int/ng-webmust/mustlink/swagger.json

MUSTLink is a REST API primarily designed to offer easy access to parameter data from MUST and ARES archives.
All endpoints are documented with detailed request/response information. Please refer to section "3. Definitions" for a list of all available models. Endpoints are also tagged and categorized according to their nature and/or behaviour, please refer to section "1.3. Tags" for a list of all available categories.

MUSTLink relies on a stateless authentication strategy where only authenticated users are able to access secured resources. It also offers additional authorization layers under the form of roles and groups. It is possible to restrict the access to a given resource to be limited to a given group or role.
The user is required to authenticate using the user authentication endpoint and obtain a JWT (JSON Web Token) token. A user token is valid for a limited period of time (applicable through the system configuration) and should be included in every secured request header with "Authorization" key. This user-specific token identifies the user to MUSTLink and serves to check user access rights for the requested resources.

MUSTLink REST endpoints are designed to provide generic responses parsable using any JSON parser. WebMUST specific endpoints are provided for GUI elements like trees, tables, buttons or links, and are accessible under /web/ path.

Contact MUST Service

Authorize

GUI

Buttons

DataProviders

GET	/web/bin/dataproviders/{ds}/parameters/data	Get binned parameter data containing the duration of samples in each bin	
GET	/web/category/dataproviders/{ds}/parameters/data	Get category parameter data	

GET `/web/category/dataproviders/{ds}/parameters/orbit` Get category parameter data including orbital data

GET `/dataproviders/{ds}/chart/png` Generates a chart in PNG format

GET `/dataproviders/{ds}/chart/svg` Generates a chart in SVG format

GET `/dataproviders` Lists the metadata of all available data providers

GET `/dataproviders/{ds}` Gets the metadata for the given Data Provider name

GET `/dataproviders/{ds}/parameters/data/paginated` Retrieves the parameter data for the given arguments with the given cursor.

Returns a list of parameters matching the given query parameters. The given data consists on time value tuples.

Parameters **Try it out**

Name	Description
ds * required `string` *(path)*	ds
key `string` *(query)*	key
values `string` *(query)*	values
from `string` *(query)*	from
to `string` *(query)*	to

Name	Description
calibrate **boolean** ***(query)***	--
aggregationFunction **string** ***(query)***	aggregationFunction
aggregation **string** ***(query)***	aggregation
aggregationValue **integer($int32)** ***(query)***	aggregationValue
compressionError **number($double)** ***(query)***	compressionError
mode **string** ***(query)***	*Available values* : SIMPLE, COMPLEX --
delta **integer($int64)** ***(query)***	delta
stream **boolean** ***(query)***	--
cursor **string** ***(query)***	cursor
limit **integer($int32)** ***(query)***	limit
compact **boolean** ***(query)***	*Default value* : false **false**

Responses

Response content type application/json

Code	Description
200	successful operation **Example Value** Model


```
            }
          ]
        },
        "metadata": [
          {
            "key": "string",
            "value": "string"
          }
        ],
        "data": [
          {
            "date": "string",
            "value": 0,
            "calibratedValue": "string"
          }
        ]
      }
    ],
    "cursor": "string"
  }
```


401	Unauthorized Request.

GET `/dataproviders/{ds}/parameters/data` Retrieves the parameter data for the given arguments.

Returns a list of parameters matching the given query parameters. The given data consists on time value tuples.

Parameters **Try it out**

Name	Description
ds * required `string` *(path)*	ds
key `string` *(query)*	key
values `string` *(query)*	values
from `string` *(query)*	from
to `string` *(query)*	to
calibrate `boolean` *(query)*	--
aggregationFunction `string` *(query)*	aggregationFunction
aggregation `string` *(query)*	aggregation
aggregationValue `integer($int32)` *(query)*	aggregationValue
compressionError `number($double)` *(query)*	compressionError

Name	Description
chunkCount **integer($int32)** ***(query)***	chunkCount
mode **string** ***(query)***	*Available values* : SIMPLE, COMPLEX --
delta **integer($int64)** ***(query)***	delta
stream **boolean** ***(query)***	--
compact **boolean** ***(query)***	*Default value* : false **false**

Responses **Response content type** **application/json**

Code	Description
200	successful operation **Example Value** Model


```
[
  {
    "monitoringChecks": {
      "requiredSamples": 0,
      "useCalibrated": true,
      "checkInterpretation": "REAL",
      "checkDefinitions": [
        {
          "position": 0,
          "type": "SOFT",
          "lowValue": "string",
          "highValue": "string",
          "checkParameter": "string",
          "checkValue": 0
```

Code	Description
401	Unauthorized Request.


```
        }
      ]
    },
    "metadata": [
      {
        "key": "string",
```


GET	**/dataproviders/{ds}/parameters/data/export**	Export the parameter data as application/zip format
GET	**/dataproviders/{ds}/parameters**	Get parameter metadata
GET	**/dataproviders/{ds}/parameters/statistics**	Gets the statistics for all the parameters matching the query arguments criteria
GET	**/dataproviders/{ds}/aggregations**	Gets the aggregation information of a dataprover
GET	**/dataproviders/{ds}/{dt}/goa**	Gets the geo reference assets for GOA
GET	**/dataproviders/geo/{dt}/goa/filters**	Filters and returns table data for GOA
GET	**/dataproviders/{ds}/orbit**	Retrieves the parameter Orbit Id and Orbit Angle for a given timestamp.
GET	**/dataproviders/{ds}/table/{dt}/export/orbit**	Gets the data of the given table {dt} as application/zip including the orbital position.
GET	**/dataproviders/{ds}/parameters/orbit**	Retrieves the parameter orbital data for the given parameters.
GET	**/dataproviders/{ds}/parameters/data/orbit/export**	Exports the parameter data with orbital data in application/zip format
GET	**/dataproviders/{ds}/table/{dt}/data**	Get the data of the given table.

GET	`/dataproviders/{ds}/table/{dt}/export`	Export the data of the given table as application/zip.
GET	`/dataproviders/{ds}/tables`	Gets a list with all available table types.
GET	`/dataproviders/{ds}/table/{dt}/metadata`	Gets the metadata of the given table.
HEAD	`/metadata/flush`	Clear DataProvider Caches
HEAD	`/metadata/flushtree`	Clear Parameter MetadataTree Caches

Parameters

GET	`/web/bin/dataproviders/{ds}/parameters/data`	Get binned parameter data containing the duration of samples in each bin
GET	`/web/category/dataproviders/{ds}/parameters/data`	Get category parameter data
GET	`/web/category/dataproviders/{ds}/parameters/orbit`	Get category parameter data including orbital data
GET	`/dataproviders/{ds}/parameters/data/paginated`	Retrieves the parameter data for the given arguments with the given cursor.
GET	`/dataproviders/{ds}/parameters/data`	Retrieves the parameter data for the given arguments.
GET	`/dataproviders/{ds}/parameters/data/export`	Export the parameter data as application/zip format
GET	`/dataproviders/{ds}/parameters`	Get parameter metadata
GET	`/dataproviders/{ds}/parameters/statistics`	Gets the statistics for all the parameters matching the query arguments criteria
GET	`/dataproviders/{ds}/aggregations`	Gets the aggregation information of a dataprover

GET	/dataproviders/{ds}/parameters/orbit	Retrieves the parameter orbital data for the given parameters.
GET	/dataproviders/{ds}/parameters/data/orbit/export	Exports the parameter data with orbital data in application/zip format

Category

GET	/web/bin/dataproviders/{ds}/parameters/data	Get binned parameter data containing the duration of samples in each bin
GET	/web/category/dataproviders/{ds}/parameters/data	Get category parameter data
GET	/web/category/dataproviders/{ds}/parameters/orbit	Get category parameter data including orbital data

Orbits

GET	/web/category/dataproviders/{ds}/parameters/orbit	Get category parameter data including orbital data
GET	/web/tables/{ds}/{dt}/orbit	Get WebMUST table data including orbital data
GET	/chart/orbit/export/png	Gets a chart image with respect to orbital position in PNG format
GET	/dataproviders/{ds}/orbit	Retrieves the parameter Orbit Id and Orbit Angle for a given timestamp.
GET	/dataproviders/{ds}/table/{dt}/export/orbit	Gets the data of the given table {dt} as application/zip including the orbital position.
GET	/dataproviders/{ds}/parameters/orbit	Retrieves the parameter orbital data for the given parameters.
GET	/dataproviders/{ds}/parameters/data/orbit/export	Exports the parameter data with orbital data in application/zip format

WebMUST

GET `/web/plugins` Get available plugins

GET `/web/tables/{ds}/{dt}` Get WebMUST table data

GET `/web/tables/params/{ds}/{dt}` Get WebMUST table parameter data

GET `/web/tables/{ds}/{dt}/orbit` Get WebMUST table data including orbital data

PATCH `/profiles/{profileName}/rename` Renames a profile with the given name

GET `/profiles/{name}` Get profile by name and username

POST `/profiles/{name}` Save a profile with the given name

DELETE `/profiles/{name}` Delete profile by name

PATCH `/profiles/visibility` Change a profile visibility by name

PUT `/profiles/share` Share a profile with the given name

GET `/profiles` Get all user accessible profiles

Plugins

GET `/web/plugins` Get available plugins

Table

GET `/web/tables/{ds}/{dt}` Get WebMUST table data

GET `/web/tables/params/{ds}/{dt}` Get WebMUST table parameter data

GET `/web/tables/{ds}/{dt}/orbit` Get WebMUST table data including orbital data

GET	`/dataproviders/geo/{dt}/goa/filters`	Filters and returns table data for GOA
GET	`/dataproviders/{ds}/table/{dt}/export/orbit`	Gets the data of the given table {dt} as application/zip including the orbital position.
GET	`/dataproviders/{ds}/table/{dt}/data`	Get the data of the given table.
GET	`/dataproviders/{ds}/table/{dt}/export`	Export the data of the given table as application/zip.
GET	`/dataproviders/{ds}/tables`	Gets a list with all available table types.
GET	`/dataproviders/{ds}/table/{dt}/metadata`	Gets the metadata of the given table.

Tree

GET	`/web/metadata/tree`	Get WebMUST tree data
GET	`/web/metadata/treesearch`	Get WebMUST search tree data
GET	`/metadata/tree/{ds}`	Get the metadata subtree.
GET	`/metadata/treesearch`	Search for tree nodes.

Dr MUST

GET	`/analysis/drmust/{taskId}`	Get a Dr MUST task by id
DELETE	`/analysis/drmust/{taskId}`	Deletes a Dr MUST task
GET	`/analysis/drmust`	Get tasks overview
POST	`/analysis/drmust`	Creates a Dr MUST task
GET	`/analysis/drmust/numresults`	Get the number of Dr MUST tasks

Novelty Detection

GET	`/analysis/novelty/missions`	Gets missions with available novelties
GET	`/analysis/{ds}/novelty`	Gets all novelties

User Management

POST	`/auth/login`	Logs user into MUSTLink
PUT	`/usermanagement/group/id/{groupId}/add/project/{projectId}`	Add project to a group by id
PUT	`/usermanagement/group/id/{groupId}/remove/project/{projectId}`	Remove project from a group by id
PUT	`/usermanagement/group/id/{groupId}/add/role/{roleId}`	Add role to a group by id
PUT	`/usermanagement/group/id/{groupId}/remove/role/{roleId}`	Remove role from a group by id
PUT	`/usermanagement/group/id/{groupId}/add/user/{userId}`	Add user to a group by id

PUT	/usermanagement/group/name/{groupName}/add/user/{userLogin}	Add user to a group by name
PUT	/usermanagement/group/id/{groupId}/set/roles/{roleIdList}	Set group roles
PUT	/usermanagement/group/name/{groupName}/add/role/{roleName}	Add role to a group by name
PUT	/usermanagement/group/id/{groupId}/set/projects/{projectIdList}	Set group projects
PUT	/usermanagement/group/name/{groupName}/add/project/{projectName}	Add project to a group by name
PUT	/usermanagement/group/id/{groupId}/remove/user/{userId}	Remove user from a group by id
PUT	/usermanagement/group/name/{groupName}/remove/user/{userLogin}	Remove user from a group by name
PUT	/usermanagement/group/name/{groupName}/remove/role/{roleName}	Remove role from a group by name
PUT	/usermanagement/group/name/{groupName}/remove/project/{projectName}	Remove project from a group by name
GET	/usermanagement/groups	Get all groups
POST	/usermanagement/groups	Create a new group
GET	/usermanagement/groups/id/{id}	Get a group by id
PUT	/usermanagement/groups/id/{id}	Update group by id
GET	/usermanagement/groups/name/{name}	Get a group by name
PUT	/usermanagement/groups/name/{name}	Update group by name

PUT `/usermanagement/groups/id/{id}/full` Update group, role list and project list by id

DELETE `/usermanagement/groups/{name}` Delete group by name

GET `/usermanagement/projects` Get all projects

POST `/usermanagement/projects` Create a project

PUT `/usermanagement/projects/name/{name}` Update project by name

DELETE `/usermanagement/projects/name/{name}` Delete project by name

PUT `/usermanagement/projects/id/{id}` Update project by id

DELETE `/usermanagement/projects/id/{id}` Delete project by id

GET `/usermanagement/roles` Get all roles

POST `/usermanagement/roles` Create a role

PUT `/usermanagement/roles/name/{name}` Update role by name

DELETE `/usermanagement/roles/name/{name}` Delete role by name

PUT `/usermanagement/roles/id/{id}` Update role by id

DELETE `/usermanagement/roles/id/{id}` Delete role by id

GET `/usermanagement/userinfo` Get information of the logged in user

GET `/usermanagement/users` Get all users

POST `/usermanagement/users` Create a new user

GET `/usermanagement/users/id/{id}` Get a user by id

GET `/usermanagement/users/login/{login}` Get a user by login name

PUT `/usermanagement/users/{login}` Update user by login name

DELETE `/usermanagement/users/{login}` Delete user

PUT `/usermanagement/users/changepassword` Change user password

Authentication

POST `/auth/login` Logs user into MUSTLink

Charting

GET `/dataproviders/{ds}/chart/png` Generates a chart in PNG format

GET `/dataproviders/{ds}/chart/svg` Generates a chart in SVG format

GET `/chart/orbit/export/png` Gets a chart image with respect to orbital position in PNG format

DataProviders Charting

GET `/chart/export/png` Generates a PNG chart image from a profile

Application Settings

GET `/appsettings` Gets application settings

Metadata

GET `/dataproviders` Lists the metadata of all available data providers

GET `/dataproviders/{ds}` Gets the metadata for the given Data Provider name

GET	`/dataproviders/{ds}/parameters`	Get parameter metadata
GET	`/dataproviders/{ds}/tables`	Gets a list with all available table types.
GET	`/dataproviders/{ds}/table/{dt}/metadata`	Gets the metadata of the given table.
HEAD	`/metadata/flush`	Clear DataProvider Caches
HEAD	`/metadata/flushtree`	Clear Parameter MetadataTree Caches

Export

GET	`/dataproviders/{ds}/parameters/data/export`	Export the parameter data as application/zip format
GET	`/chart/orbit/export/png`	Gets a chart image with respect to orbital position in PNG format
GET	`/dataproviders/{ds}/table/{dt}/export/orbit`	Gets the data of the given table {dt} as application/zip including the orbital position.
GET	`/dataproviders/{ds}/parameters/data/orbit/export`	Exports the parameter data with orbital data in application/zip format
GET	`/dataproviders/{ds}/table/{dt}/export`	Export the data of the given table as application/zip.

Statistics

GET	`/dataproviders/{ds}/parameters/statistics`	Gets the statistics for all the parameters matching the query arguments criteria

Aggregations

GET	`/dataproviders/{ds}/aggregations`	Gets the aggregation information of a dataprover

GOA

GET `/dataproviders/{ds}/{dt}/goa` Gets the geo reference assets for GOA

GET `/dataproviders/geo/{dt}/goa/filters` Filters and returns table data for GOA

Highlighted Conditions

PATCH `/hc/set/name/{id}` Rename a highlighted condition by id

PATCH `/hc/set/name` Rename a highlighted condition by name

GET `/hc/all` Get all accessible highlighted conditions

GET `/hc/{id}` Gets a highlighted condition by id

PUT `/hc/{id}` Updates a highlighted condition by id

DELETE `/hc/{id}` Deletes a highlighted condition by id

GET `/hc/username/{username}` Gets a highlighted condition by name

PATCH `/hc/set/public/{id}` Update a highlighted condition's public attribute by id

PATCH `/hc/set/public` Update highlighted condition's public attribute by name

PATCH `/hc/share` Shares a highlighted condition by name

PATCH `/hc/share/{id}` Shares a highlighted condition by id

POST `/hc` Create a new highlighted condition

PUT `/hc` Update a highlighted condition by name

DELETE `/hc` Delete highlighted condition by name

Profiles

PATCH	`/profiles/{profileName}/rename`	Renames a profile with the given name
GET	`/profiles/{name}`	Get profile by name and username
POST	`/profiles/{name}`	Save a profile with the given name
DELETE	`/profiles/{name}`	Delete profile by name
PATCH	`/profiles/visibility`	Change a profile visibility by name
PUT	`/profiles/share`	Share a profile with the given name
GET	`/profiles`	Get all user accessible profiles

MUSE

POST	`/scripts/adhoc/run`	Executes ad hoc script
GET	`/{ds}/scripts/{id}/run`	Executes a saved script
PUT	`/scripts/set/name`	Rename a script by name
PUT	`/scripts/set/public`	Update script public attribute by name
GET	`/scripts`	Get all accessible scripts
POST	`/scripts`	Create a new script
PUT	`/scripts`	Update a script by name
DELETE	`/scripts`	Delete script by name

GET `/scripts/{id}` Get script by id

PUT `/scripts/{id}` Update a script by id

DELETE `/scripts/{id}` Delete script by id

GET `/scripts/username/{username}` Get script by name

PUT `/scripts/set/name/{id}` Rename a script by id

PUT `/scripts/set/public/{id}` Update script public attribute by id

PUT `/scripts/share/{id}` Share a script by id

PUT `/scripts/share` Share a script by name

PUT `/scripts/set/store/{id}` Update store attribute by id

PUT `/scripts/set/store` Update store attribute by name

UserContent

POST `/content` Create a new user content

PUT `/content` Update content by name and type

GET `/content/{id}` Get content by id

DELETE `/content/{id}` Delete content by id

GET `/content/name` Get content by id

GET `/content/type/{type}` Get all accessible content of a given type

PUT	`/content/set/name/{id}`	Rename a content by id
PUT	`/content/set/public/{id}`	Update content public attribute by id
PUT	`/content/share/{id}`	Share a content by id
PUT	`/content/share/name`	Share a content by name and type

Groups

PUT	`/usermanagement/group/id/{groupId}/add/project/{projectId}`	Add project to a group by id
PUT	`/usermanagement/group/id/{groupId}/remove/project/{projectId}`	Remove project from a group by id
PUT	`/usermanagement/group/id/{groupId}/add/role/{roleId}`	Add role to a group by id
PUT	`/usermanagement/group/id/{groupId}/remove/role/{roleId}`	Remove role from a group by id
PUT	`/usermanagement/group/id/{groupId}/add/user/{userId}`	Add user to a group by id
PUT	`/usermanagement/group/name/{groupName}/add/user/{userLogin}`	Add user to a group by name
PUT	`/usermanagement/group/id/{groupId}/set/roles/{roleIdList}`	Set group roles
PUT	`/usermanagement/group/name/{groupName}/add/role/{roleName}`	Add role to a group by name
PUT	`/usermanagement/group/id/{groupId}/set/projects/{projectIdList}`	Set group projects
PUT	`/usermanagement/group/name/{groupName}/add/project/{projectName}`	Add project to a group by name

PUT	`/usermanagement/group/id/{groupId}/remove/user/{userId}`	Remove user from a group by id
PUT	`/usermanagement/group/name/{groupName}/remove/user/{userLogin}`	Remove user from a group by name
PUT	`/usermanagement/group/name/{groupName}/remove/role/{roleName}`	Remove role from a group by name
PUT	`/usermanagement/group/name/{groupName}/remove/project/{projectName}`	Remove project from a group by name
GET	`/usermanagement/groups`	Get all groups
POST	`/usermanagement/groups`	Create a new group
GET	`/usermanagement/groups/id/{id}`	Get a group by id
PUT	`/usermanagement/groups/id/{id}`	Update group by id
GET	`/usermanagement/groups/name/{name}`	Get a group by name
PUT	`/usermanagement/groups/name/{name}`	Update group by name
PUT	`/usermanagement/groups/id/{id}/full`	Update group, role list and project list by id
DELETE	`/usermanagement/groups/{name}`	Delete group by name

Projects

GET	`/usermanagement/projects`	Get all projects
POST	`/usermanagement/projects`	Create a project
PUT	`/usermanagement/projects/name/{name}`	Update project by name

DELETE `/usermanagement/projects/name/{name}` Delete project by name

PUT `/usermanagement/projects/id/{id}` Update project by id

DELETE `/usermanagement/projects/id/{id}` Delete project by id

Roles

GET `/usermanagement/roles` Get all roles

POST `/usermanagement/roles` Create a role

PUT `/usermanagement/roles/name/{name}` Update role by name

DELETE `/usermanagement/roles/name/{name}` Delete role by name

PUT `/usermanagement/roles/id/{id}` Update role by id

DELETE `/usermanagement/roles/id/{id}` Delete role by id

Models

ButtonModel

GUIModel

TopLinksModel

BinModel

MetadataPropertyModel

ParameterBinsModel

CategorySeriesModel

IntervalModel

ParameterCategoryModel

CategoryOrbitDataPointModel

CategoryOrbitSeriesModel

OrbitParameterCategoryModel

PluginsModel

ParameterTableColumnHeader

TableCellModel

TableDataModel

TableDetailModel

TableFilterModel

TableModel

OrbitTableModel

TreeNodeAttributesModel

TreeNodeDataModel

TreeNodeMetadataLinkModel

TreeNodeMetadataModel

TreeNodeModel

TreeNodeSearchModel

CorrelatorTaskResult

DrMustResultModel

FeatureResult

GidResult

ParameterResult

ShapeResultPeriod

ShapeTaskResult

TimePeriodResult

DrMustTaskOverviewModel

DrMUSTTaskNumberModel

DrMustTaskModel

MissionModel

NoveltyModel

JsonWebToken

CredentialsModel

ApplicationSettingsModel

DataProviderModel

MonitoringCheckDefinition

MonitoringChecks

NumericDataPointModel

PaginatedParameterModel

ParameterModel

StatisticsModel

AggregationsBaseModel

AggregationsModel

GeoAsset

GeoAssetPropertyModel

GeoPoint

TablesOverviewModel

TableMetadataModel

HighlightedConditionUserModel

HighlightedConditionModel

HighlightedConditionPostModel

Link

TreeNode

SearchResultNodeModel

Profile

MetaProfileModel

AdhocScriptModel

ScriptUserModel

ScriptModel

UserContentModel

GroupModel

ProjectModel

RoleModel

UserModel

